

CESUPA — Centro Universitário do Estado do Pará

Inteligência Artificial e Computacional

Prof. Daniel Leal Souza

Turma: CC5NA

Semestre: 01/2026

Helmsman

Fuzzy Auto-Scaler for Docker Web Services

com modo Algoritmo Genético integrado

Andrey de Matos Gonçalves

Felipe Gabriel Souza Libório

Felipe Vieira Brazão e Silva

Gabriel Coelho Góes

Belém, PA

Junho de 2026

Resumo

Este trabalho apresenta o **Helmsman**, uma biblioteca Python com interface de linha de comando que implementa um sistema de escalonamento automático de containers Docker com dois modos de operação: lógica fuzzy Mamdani (modo padrão) e Algoritmo Genético (modo `-mode ag`). O problema abordado é a limitação dos auto-scalers tradicionais, como o *Horizontal Pod Autoscaler* (HPA) do Kubernetes, que operam com limiares binários rígidos e tomam decisões sem contexto. O modo fuzzy combina três métricas normalizadas — uso de CPU, uso de RAM e requisições por segundo — em 22 regras linguísticas organizadas em 7 grupos funcionais, com defuzzificação por centróide. O modo AG implementa um algoritmo genético do zero, sem bibliotecas de otimização, que evolui uma população de cromossomos `[n_replicas, sla_threshold]` para encontrar a configuração ótima a cada ciclo de monitoramento. Ambos os modos foram validados com os mesmos 12 cenários de teste: o fuzzy obteve aprovação em 12/12, e o AG em 55/60 execuções (91%) considerando 5 sementes distintas. Os resultados demonstram que a abordagem fuzzy é mais determinística e precisa para casos extremos, enquanto o AG apresenta precisão elevada e flexibilidade adaptativa ao otimizar simultaneamente o número de réplicas e o limiar de SLA.

Palavras-chave: Lógica Fuzzy, Mamdani, Algoritmo Genético, Auto-scaling, Docker, Computação Evolutiva.

Sumário

1	Introdução	4
1.1	Objetivos	4
1.2	Justificativa para uso de computação inteligente	5
2	Fundamentação Teórica	5
2.1	Conjuntos fuzzy e funções de pertinência	5
2.2	Sistema de inferência Mamdani	6
2.3	Algoritmos Genéticos	6
2.4	Trabalhos relacionados	7
3	Metodologia e Modelagem	7
3.1	Arquitetura do sistema	7
3.2	Variáveis do sistema	8
4	Modo Fuzzy Mamdani	9
4.1	Funções de pertinência	9
4.2	Base de regras	10
4.2.1	Grupo 1 — Sistema ocioso, scale down	10
4.2.2	Grupo 2 — RPS alto, recursos livres	10
4.2.3	Grupo 3 — Carga real, scale up normal	11
4.2.4	Grupo 4 — Carga crítica, scale up urgente	11
4.2.5	Grupo 5 — Host cheio, bloqueia scale up	11
4.2.6	Grupo 6 — Anomalias operacionais	12
4.2.7	Grupo 7 — RPS crítico com recursos livres	12
5	Modo Algoritmo Genético	13
5.1	Formulação do problema	13
5.2	Função de aptidão	13
5.3	Parâmetros e operadores	14
5.4	Custo computacional	15
5.5	Extensão: Integração Fuzzy-Evolutiva	15
5.5.1	Comparação de três abordagens	16
5.5.2	Análise de impacto	16
6	Implementação	17
6.1	Stack tecnológica	17
6.2	Estrutura do repositório	17
7	Experimentos e Resultados	18

7.1	Cenários de teste — modo fuzzy	18
7.2	Cenários de teste — modo AG	19
7.3	Superfície de controle — modo fuzzy	19
7.4	Comparação entre os dois modos	20
7.5	Análise crítica	20
8	Conclusão	21
	Declaração de Uso de Inteligência Artificial	21

1 Introdução

O escalonamento automático de serviços web é um problema central em infraestrutura moderna. A abordagem dominante consiste em definir limiares fixos para métricas como CPU ou memória e disparar ações quando esses limiares são ultrapassados. Essa estratégia, adotada pelo HPA do Kubernetes e por ferramentas similares como o KEDA [7] e o Prometheus Adapter [8], apresenta uma limitação fundamental: as decisões são tomadas sem considerar o contexto formado pelo conjunto das métricas.

Um servidor com 85% de CPU e zero requisições por segundo não está sob carga — está com um processo travado ou vazamento de recursos. Escalar réplicas nesse cenário não resolve o problema e ainda consome recursos desnecessários do *host*. Da mesma forma, um servidor com 79% de CPU e um com 81% não devem gerar respostas radicalmente diferentes, como ocorre em sistemas de limiar rígido. Em ambientes com grande capacidade computacional — dezenas de vCPUs e centenas de gigabytes de RAM — determinar o número ideal de containers a cada momento, considerando demanda variável e múltiplos critérios conflitantes, é um problema de otimização genuíno que não admite solução manual em tempo real.

A lógica fuzzy, introduzida por Zadeh [1] e aplicada a sistemas de controle por Mamdani e Assilian [2], oferece uma abordagem alternativa: permite expressar decisões graduais e combinar múltiplos sinais antes de agir. Algoritmos Genéticos [3], por sua vez, são metaheurísticas bioinspiradas capazes de explorar espaços de busca multidimensionais de forma adaptativa, sendo adequados para otimização de configurações com critérios conflitantes.

Este trabalho apresenta o **Helmsman**, um sistema de escalonamento para containers Docker em host único, distribuído como biblioteca Python instalável via `pip`, que oferece dois modos de operação selecionáveis por CLI: o modo fuzzy Mamdani e o modo Algoritmo Genético.

1.1 Objetivos

- Implementar um sistema de controle fuzzy Mamdani para escalonamento de containers Docker;
- Modelar a base de regras a partir de conhecimento de domínio de infraestrutura;
- Detectar anomalias operacionais (*memory leak*, *CPU leak*) como subproduto da inferência;
- Implementar um Algoritmo Genético do zero como modo alternativo de decisão;
- Comparar os dois modos nos mesmos cenários de teste com métricas quantitativas.

1.2 Justificativa para uso de computação inteligente

Cinco razões justificam o uso de lógica fuzzy e computação evolutiva neste problema. Primeira: as métricas de infraestrutura são inerentemente contínuas — não há fronteira exata entre “carga moderada” e “carga alta”. Segunda: o contexto formado pela combinação de sinais determina a ação correta, não o valor isolado de cada métrica. Terceira: o conhecimento de operadores de infra é naturalmente linguístico. Quarta: otimizar simultaneamente número de réplicas e limiar de SLA com critérios conflitantes (custo vs disponibilidade) é um problema combinatório adequado para metaheurísticas. Quinta: em hosts com grande capacidade, a força bruta não escala — o AG explora o espaço de busca de forma inteligente.

Tabela 1: Comparação entre o HPA tradicional e o Helmsman

Situação	HPA tradicional	Helmsman
CPU 85% com zero requisições	Escala (incorreto)	Detecta anomalia, não escala
RAM crítica com demanda alta	Escala (pode derrubar o host)	Bloqueia scale up, emite alerta crítico
RPS alto com CPU baixa	Escala (desnecessário)	Avalia contexto completo
CPU em 79% vs 81%	Decisões completamente diferentes	Transição suave e gradual
Detecção de <i>leak</i>	Não detecta	Alerta via padrão RPS baixo + recurso alto
Configuração	YAML complexo + kube-config	<code>pip install</code> + um comando CLI

2 Fundamentação Teórica

2.1 Conjuntos fuzzy e funções de pertinência

Na teoria dos conjuntos clássicos, um elemento pertence ou não pertence a um conjunto. Zadeh [1] propôs uma generalização em que a pertinência é graduada no intervalo $[0, 1]$. Para um universo de discurso X , um conjunto fuzzy A é definido por sua função de pertinência $\mu_A : X \rightarrow [0, 1]$, onde $\mu_A(x)$ representa o grau com que x pertence a A .

O Helmsman utiliza dois tipos de funções de pertinência:

Função trapezoidal (`trapmf`), parametrizada por $[a, b, c, d]$:

$$\mu(x) = \begin{cases} 0 & x \leq a \text{ ou } x \geq d \\ \frac{x-a}{b-a} & a < x < b \\ 1 & b \leq x \leq c \\ \frac{d-x}{d-c} & c < x < d \end{cases} \quad (1)$$

Função triangular (`trimf`), parametrizada por $[a, b, c]$:

$$\mu(x) = \begin{cases} 0 & x \leq a \text{ ou } x \geq c \\ \frac{x-a}{b-a} & a < x \leq b \\ \frac{c-x}{c-b} & b < x < c \end{cases} \quad (2)$$

2.2 Sistema de inferência Mamdani

O sistema de inferência Mamdani [2] opera em quatro etapas:

1. **Fuzzificação:** converte os valores nítidos das entradas em graus de pertinência nas funções definidas;
2. **Avaliação das regras:** para cada regra com antecedentes $A_1, A_2, \dots, A_n$ conectados por AND, o grau de ativação é $\alpha = \min(\mu_{A_1}(x_1), \mu_{A_2}(x_2), \dots, \mu_{A_n}(x_n))$;
3. **Agregação:** o consequente de cada regra é truncado ao grau α (implicação mínimo); os consequentes de todas as regras são combinados por máximo;
4. **Defuzzificação:** o conjunto fuzzy agregado é convertido em valor nítido pelo método do centróide:

$$z^* = \frac{\int z \cdot \mu_{\text{agregado}}(z) dz}{\int \mu_{\text{agregado}}(z) dz} \quad (3)$$

2.3 Algoritmos Genéticos

Algoritmos Genéticos [3] são metaheurísticas bioinspiradas que simulam o processo de seleção natural para resolver problemas de otimização. Uma população de soluções candidatas (indivíduos) evolui ao longo de gerações por meio de três operadores: seleção dos mais aptos, recombinação (*crossover*) de características entre pares de indivíduos, e mutação aleatória para manter diversidade genética.

A adequação de cada indivíduo ao problema é medida por uma função de aptidão (*fitness*). O AG minimiza ou maximiza essa função ao longo das gerações, convergindo para regiões de melhor qualidade no espaço de busca. Para problemas estocásticos, múltiplas execuções com sementes diferentes garantem robustez estatística dos resultados.

2.4 Trabalhos relacionados

Zeng *et al.* [4] propuseram um controlador fuzzy para alocação dinâmica de recursos em servidores web, demonstrando que a abordagem fuzzy reduz violações de SLA em comparação com controladores PID. Pandey *et al.* [5] aplicaram lógica fuzzy a auto-scaling em ambientes de nuvem, com resultados superiores ao escalonamento por limiar. Dang *et al.* [6] combinaram lógica fuzzy com previsão de carga para antecipar decisões de escalonamento. Nenhum desses trabalhos aborda a detecção de anomalias operacionais como subproduto da inferência, nem compara diretamente abordagens fuzzy e evolutivas no mesmo sistema.

3 Metodologia e Modelagem

3.1 Arquitetura do sistema

O Helmsman é distribuído como biblioteca Python instalável via `pip`. Dois modos de operação são selecionáveis por CLI:

```
1 # Modo padr o      fuzzy Mamdani
2 helmsman start --host localhost --port 8000 --container meu-
   servico
3
4 # Modo AG      Algoritmo Gen tico
5 helmsman start --host localhost --port 8000 --container meu-
   servico --mode ag
```

Listing 1: Modos de operação do Helmsman

Em ambos os modos, a coleta de métricas e as ações de escalonamento são idênticas. O que muda é o motor de decisão que transforma métricas em delta de réplicas. A arquitetura geral do sistema é apresentada na Figura 1.

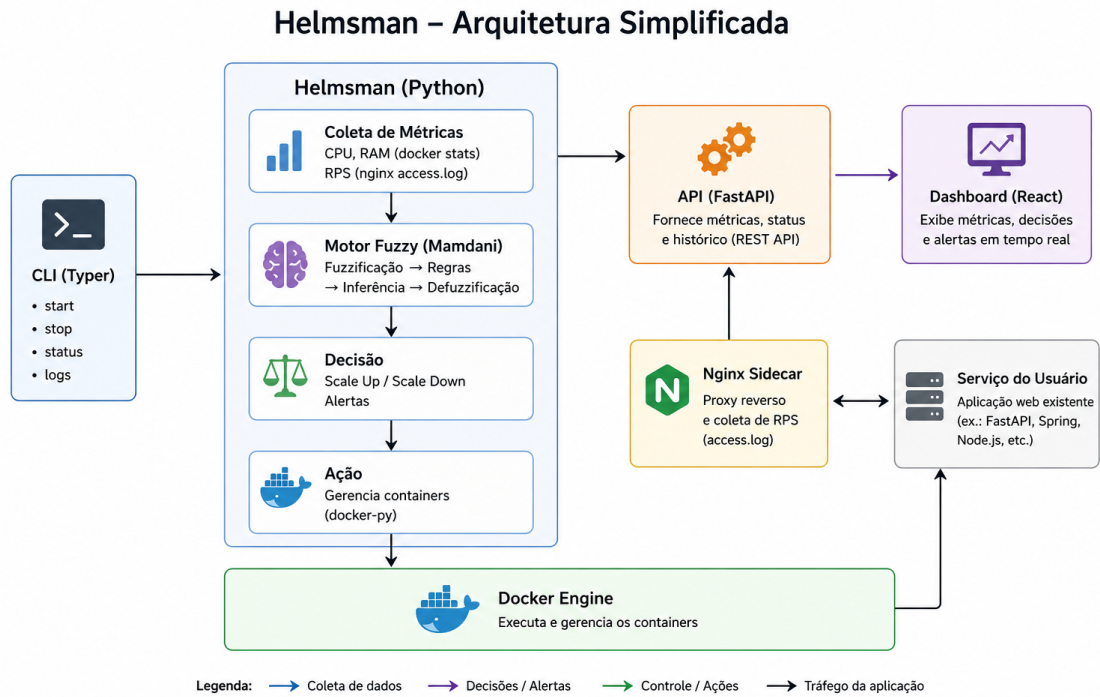


Figura 1: Arquitetura simplificada do Helmsman.

O proxy Nginx sidecar é gerado automaticamente via template Jinja2. O serviço monitorado não é modificado. A distribuição de tráfego entre réplicas é gerenciada pelo bloco **upstream** do Nginx, que é regenerado e recarregado a cada decisão de escalonamento. O reload é gracioso — conexões em andamento não são interrompidas.

3.2 Variáveis do sistema

As três entradas são normalizadas no universo $[0, 100]\%$, garantindo genericidade para qualquer host e serviço.

Tabela 2: Variáveis de entrada do sistema

Variável	Universo	Como é calculada	Fonte
cpu_pct	$[0, 100]\%$	$\text{cpu_containers} / \text{cpu_host} \times 100$	docker stats
ram_pct	$[0, 100]\%$	$\text{ram_containers} / \text{ram_host} \times 100$	docker stats
rps_pct	$[0, 100]\%$	$\text{rps_atual} / (\text{réplicas} \times \text{rps_per_replica}) \times 100$	nginx access.log

4 Modo Fuzzy Mamdani

4.1 Funções de pertinência

As três entradas compartilham os mesmos quatro termos linguísticos e parâmetros, pois operam no mesmo universo normalizado.

Tabela 3: Parâmetros das funções de pertinência das entradas

Termo	Tipo	Parâmetros	Justificativa
Baixo	Trapezoidal	[0, 0, 20, 40]	Abaixo de 20% é confortável; até 40% ainda aceitável
Moderado	Triangular	[20, 45, 70]	Zona de atenção com pico em 45%
Alto	Triangular	[55, 72, 88]	Pressão real; sobreposição intencional com vizinhos
Crítico	Trapezoidal	[75, 88, 100, 100]	Acima de 88% é emergência; platô até 100%

A sobreposição entre Alto e Moderado na faixa 55–70% e entre Alto e Crítico na faixa 75–88% é intencional, garantindo decisões graduais em vez de saltos abruptos.

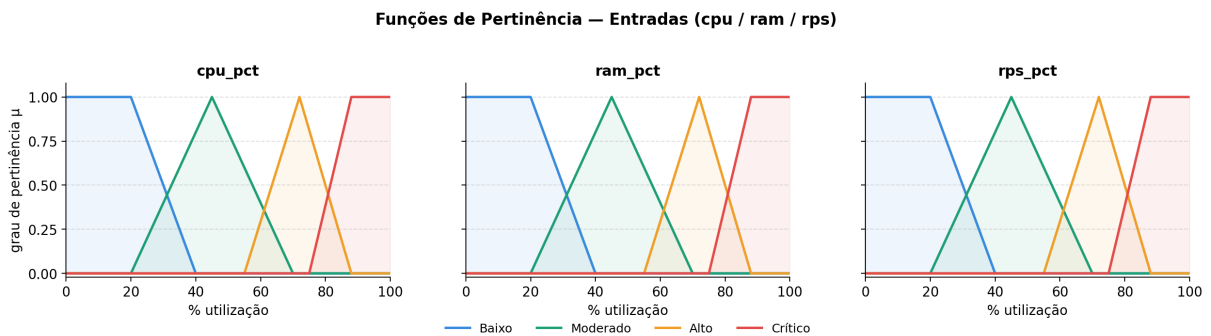


Figura 2: Funções de pertinência das variáveis de entrada.

Tabela 4: Parâmetros das funções de pertinência das saídas

Variável	Termo	Tipo	Parâmetros	Ação
	Derrubar forte	Triangular	[−3, −2, −1]	Remove 2 réplicas
	Derrubar	Triangular	[−2, −1, 0]	Remove 1 réplica
	Manter	Triangular	[−1, 0, 1]	Nenhuma ação
	Subir	Triangular	[0, 1, 2]	Adiciona 1 réplica
	Subir forte	Triangular	[1, 3, 5]	Adiciona 2–3 réplicas
	Nenhum	Trapezoidal	[0, 0, 20, 40]	Sem log especial
	Warning	Triangular	[20, 50, 80]	Log de aviso
	Critical	Trapezoidal	[60, 80, 100, 100]	Notificação imediata

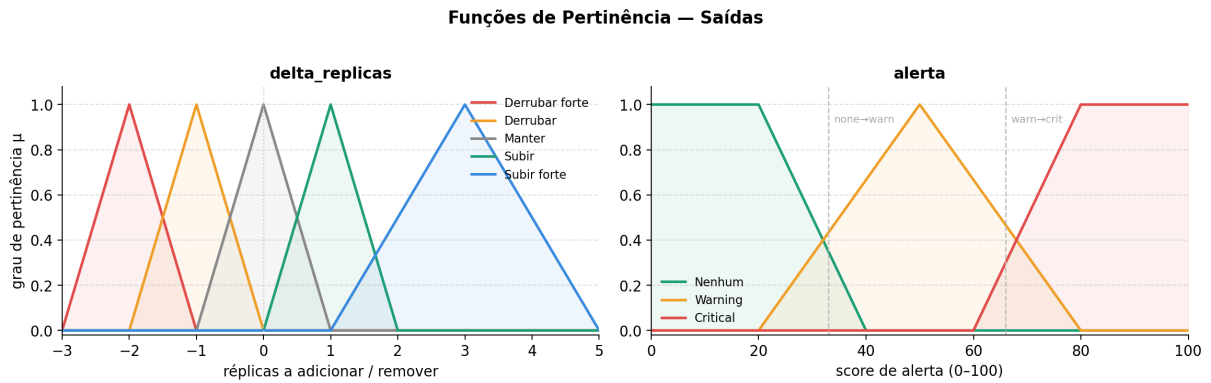


Figura 3: Funções de pertinência das variáveis de saída.

4.2 Base de regras

O sistema implementa 22 regras organizadas em 7 grupos funcionais. AND entre antecedentes = mínimo; implicação = mínimo; agregação = máximo.

4.2.1 Grupo 1 — Sistema ocioso, scale down

#	cpu	ram	rps	Operação	delta	Justificativa
1	Baixo	Baixo	Baixo	AND	Derrubar forte	Completamente ocioso
2	Baixo	Baixo	Moderado	AND	Manter	Ainda há tráfego, não derruba
3	Moderado	Baixo	Baixo	AND	Derrubar	CPU moderada sem demanda

4.2.2 Grupo 2 — RPS alto, recursos livres

#	cpu	ram	rps	Operação	delta	Justificativa
4	Baixo	Baixo	Alto	AND	Subir	Atingiu o limite de rps_per_replica
5	Baixo	Moderado	Alto	AND	Subir	Idem, RAM já crescendo

4.2.3 Grupo 3 — Carga real, scale up normal

#	cpu	ram	rps	Operação	delta	alerta	Justificativa
6	Moderado	Moderado	Alto	AND	Subir	Nenhum	Pressão confirmada nos três sinais
7	Alto	Moderado	Alto	AND	Subir	Nenhum	CPU alta com demanda alta
8	Moderado	Alto	Alto	AND	Subir	Warning	RAM alta com demanda crescente

4.2.4 Grupo 4 — Carga crítica, scale up urgente

#	cpu	ram	rps	Operação	delta	alerta	Justificativa
9	Alto	Alto	Alto	AND	Subir forte	Warning	Saturação generalizada
10	Crítico	Moderado	Alto	AND	Subir forte	Warning	CPU crítica com demanda alta
11	Alto	Moderado	Crítico	AND	Subir forte	Critical	Demanda crítica no limite

4.2.5 Grupo 5 — Host cheio, bloqueia scale up

#	cpu	ram	rps	Operação	delta	alerta	Justificativa
12	Alto	Crítico	Alto	AND	Manter	Critical	Subir container poderia derrubar o host
13	Crítico	Crítico	Crítico	AND	Manter	Critical	Colapso iminente — intervenção humana

4.2.6 Grupo 6 — Anomalias operacionais

#	cpu	ram	rps	Operação	delta	alerta	Justificativa
14	Alto	Baixo	Baixo	AND	Manter	Warning	Suspeita de CPU leak
15	Crítico	Baixo	Baixo	AND	Derrubar	Critical	CPU crítica sem demanda
16	Baixo	Alto	Baixo	AND	Derrubar	Warning	Suspeita de memory leak
17	Baixo	Crítico	Baixo	AND	Derrubar forte	Critical	Memory leak severo
18	Alto	Alto	Baixo	AND	Derrubar	Critical	CPU e RAM altas sem demanda

4.2.7 Grupo 7 — RPS crítico com recursos livres

#	cpu	ram	rps	Operação	delta	alerta	Justificativa
19	Baixo	Baixo	Crítico	AND	Subir	Warning	Capacidade esgotada, recursos livres
20	Moderado	Baixo	Crítico	AND	Subir forte	Warning	CPU crescendo + capacidade no limite
21	Baixo	Moderado	Crítico	AND	Subir	Warning	RAM crescendo + capacidade no limite
22	Moderado	Moderado	Crítico	AND	Subir forte	Warning	Tudo crescendo, escala agressiva

5 Modo Algoritmo Genético

5.1 Formulação do problema

No modo AG, o problema de escalonamento é formulado como otimização combinatória: dado o estado atual do sistema, encontrar a configuração de réplicas e limiar de SLA que minimiza uma função de aptidão multidimensional.

Variáveis de decisão (cromossomo com 2 genes):

- Gene 1: $n_replicas \in [\min_replicas, \max_replicas]$ — número de réplicas ativas
- Gene 2: $sla_threshold \in [70, 95]$ — limiar de ocupação (%) a partir do qual há sobrecarga

A Figura 4 ilustra o fluxo completo do algoritmo.

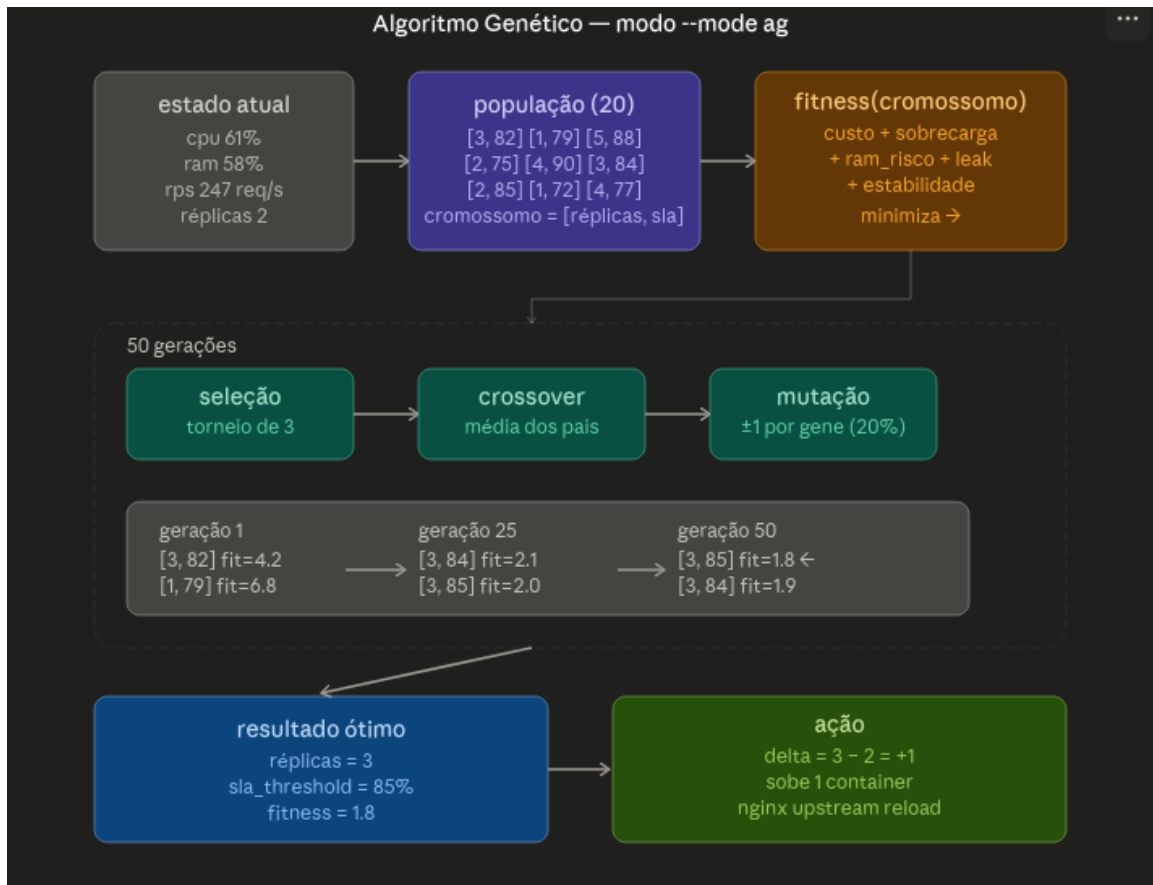


Figura 4: Fluxo do Algoritmo Genético no Helmsman.

5.2 Função de aptidão

A função de aptidão captura os mesmos cenários operacionais das regras fuzzy, porém em forma matemática. O AG minimiza:

$$f(c) = f_{\text{custo}} + f_{\text{sobrecarga}} + f_{\text{rps_abs}} + f_{\text{ram}} + f_{\text{leak}} + f_{\text{estab}} + f_{\text{sla_risco}} + f_{\text{conserv}} \quad (4)$$

onde cada termo penaliza um aspecto do problema:

Tabela 5: Componentes da função de aptidão do AG

Componente	Significado	Fórmula simplificada
Custo	Réplicas desnecessárias são caras	$n_replicas \times 1,0$
Sobrecarga	RPS acima do limiar viola SLA	$\max(0, rps_pct - \theta)^{1,5} \times 20$
RPS absoluto	Carga alta em absoluto pressiona escala	$\max(0, rps - 0,7 \times cap) \times 0,12$
RAM risco	Subir réplica com RAM alta é perigoso	$f(ram_pct) \times \max(0, \Delta) \times 8$
Leak	CPU alta sem demanda = anomalia	$cpu_alta \times sem_demanda \times 20$
Estabilidade	Evita mudanças bruscas	$ \Delta \times 0,5$
SLA risco	Threshold alto demais é perigoso	$\max(0, \theta - 90) \times 0,5$
Conservadorismo	Threshold baixo demais escala cedo	$\max(0, 75 - \theta) \times 0,3$

onde $\theta = sla_threshold/100$ e $\Delta = n_replicas - \text{réplicas_atuais}$.

5.3 Parâmetros e operadores

Tabela 6: Parâmetros do Algoritmo Genético

Parâmetro	Valor	Justificativa
Tamanho da população	20 indivíduos	Equilíbrio entre diversidade e custo
Número de gerações	50	Suficiente para convergência em espaço pequeno
Seleção	Torneio de 3	Pressão seletiva moderada, sem elitismo excessivo
Crossover	Média dos pais	Natural para genes inteiros interdependentes
Taxa de mutação	20% por gene	Mantém diversidade sem instabilizar convergência
Critério de parada	50 gerações	Fixo para garantir reprodutibilidade

O AG é implementado do zero em Python puro, usando apenas o módulo `random` da biblioteca padrão. Nenhuma biblioteca de otimização é utilizada.

5.4 Custo computacional

Com 20 indivíduos e 50 gerações, o AG realiza 1.000 avaliações de aptidão por ciclo. Cada avaliação é aritmética pura — sem I/O, sem chamadas de sistema. O tempo total por ciclo é inferior a 1ms, bem abaixo do intervalo de polling de 5 segundos.

5.5 Extensão: Integração Fuzzy-Evolutiva

Como extensão do modo AG, foi implementada uma segunda variante da função de aptidão — denominada **AG-fuzzy** — que substitui parte dos termos heurísticos originais por uma chamada direta ao motor de inferência Mamdani (Seção 2.2, mesmas 22 regras). Para cada cromossomo `[n_replicas, sla_threshold]` avaliado, simula-se a ocupação resultante caso aquele número de réplicas fosse adotado:

$$\text{sim_rps_pct} = \text{clip} \left(\frac{\text{rps_real}}{n_replicas \times \text{rps_per_replica}} \times 100, 0, 100 \right) \quad (5)$$

Esse valor simulado, junto com `cpu_pct` e `ram_pct` reais, é passado ao motor fuzzy: `infer(cpu_pct, ram_pct, sim_rps_pct)`, que retorna um `alert_score` $\in [0, 100]$. A nova função de aptidão é:

$$f_{\text{fuzzy}}(c) = f_{\text{custo}} + f_{\text{risco_fuzzy}} + f_{\text{sla}} + f_{\text{instab}} + f_{\text{conserv}} \quad (6)$$

Tabela 7: Componentes da função de aptidão AG-fuzzy

Componente	Significado	Fórmula
Custo	Réplicas desnecessárias são caras	$n_replicas \times 1,1$
Risco fuzzy	Risco do cenário simulado segundo o motor Mamdani	$\text{alert_score} \times 1,4$
SLA	Penaliza ocupação simulada acima do limiar	$\max(0, \text{sim_rps_pct} - \theta) \times 10,0$
Instabilidade	Evita mudanças bruscas	$ \Delta \times 0,8$
Conservadorismo	Penaliza thresholds extremos	ver Eq. 7

$$f_{\text{conserv}} = \begin{cases} (\theta - 90) \times 2,0 & \theta > 90 \\ (75 - \theta) \times 1,5 & \theta < 75 \\ 0 & \text{caso contrário} \end{cases} \quad (7)$$

onde $\theta = \text{sla_threshold}$ e $\Delta = n_replicas - \text{réplicas_atuais}$. Os operadores

genéticos (seleção por torneio, crossover por média, mutação de 20%) são idênticos aos da Seção 5.3.

5.5.1 Comparação de três abordagens

Para avaliar o impacto da integração fuzzy-evolutiva, três abordagens foram comparadas nos mesmos 12 cenários $\times$ 5 sementes (60 execuções), implementadas em `tests/ag_comparison.py`:

- **Heurística simples** — regra fixa baseada apenas em `rps_pct` atual (sem AG, sem fuzzy), representando um auto-scaler de limiar rígido tipo HPA;
- **AG-puro** — AG com a função de aptidão original (Seção 5.2), sem componente fuzzy;
- **AG-fuzzy** — AG com a função de aptidão híbrida descrita acima.

Tabela 8: Comparação entre heurística, AG-puro e AG-fuzzy — 60 execuções

Método	Aprovados	Falhas	Taxa
Heurística simples (sem otimização)	25/60	35	41%
AG-puro (fitness original)	55/60	5	91%
AG-fuzzy (Alternativa 2)	45/60	15	75%

5.5.2 Análise de impacto

A integração fuzzy-evolutiva supera amplamente a heurística de limiar fixo (75% vs. 41%), confirmando que mesmo uma fitness híbrida ainda captura muito mais contexto do que regras binárias. Entretanto, em relação ao AG-puro, a integração fuzzy resultou em uma **regressão de 16 pontos percentuais** (91% $\rightarrow$ 75%).

A análise cenário a cenário mostra um *trade-off*, não uma piora uniforme:

- **T3 (carga real crescente)** — passou a ser aprovado com a fitness fuzzy, pois o `alert_score` captura corretamente a combinação de CPU e RAM moderadas com RPS alto, algo que a fitness original subestimava;
- **T4 (pico de tráfego), T10 (scale down gradual) e T11 (RPS crítico, CPU baixa)** — passaram a falhar. Em todos os três casos, o `sim_rps_pct` recalculado para o cromossomo candidato cai rapidamente assim que o número de réplicas aumenta, fazendo o `alert_score` do motor fuzzy oscilar entre níveis de alerta vizinhos (*none/warning*) de forma pouco discriminante para esses estados — o termo fuzzy não consegue, sozinho, distinguir o candidato ótimo dos vizinhos com a mesma precisão que os termos analíticos originais (`rps_absoluto`, `sobrecarga`).

Em suma, o motor fuzzy contribui com uma penalidade de risco mais "contextual"(vantajosa em T3), mas, como usado aqui — recalculado a partir de um `n_replicas` candidato em constante mudança — perde a granularidade necessária para os cenários de fronteira onde o AG-puro já era preciso. Isso evidencia uma limitação conhecida de fitness híbridas: combinar dois sistemas que já são individualmente bem ajustados (regras fuzzy calibradas para o estado *real*, função analítica calibrada para o estado *candidato*) não garante que a combinação supere ambos — o ponto de acoplamento entre os dois (aqui, o recálculo de `sim_rps_pct`) é o fator decisivo.

Trabalho futuro: usar o `alert_score` do estado *atual* (não simulado) como um termo de contexto fixo somado a todos os candidatos — em vez de recalculado por candidato —, preservando a precisão dos termos analíticos para diferenciar `n_replicas` e usando o fuzzy apenas como sinalizador de urgência geral do cenário.

6 Implementação

6.1 Stack tecnológica

Tabela 9: Stack tecnológica do Helmsman

Componente	Tecnologia	Justificativa
CLI	Python + Typer	Interface limpa, geração automática de <code>--help</code>
Motor fuzzy	scikit-fuzzy	Implementação Mamdani consolidada
Motor AG	Python puro	Implementação do zero, sem libs de otimização
Coleta CPU/RAM	docker-py	API oficial do Docker em Python
Coleta RPS	nginx access.log	Sem modificação no serviço alvo
Scale up/down	docker-py	Gerenciamento programático de containers
Proxy sidecar	Nginx + Jinja2	Template gerado a partir dos parâmetros da CLI
API backend	FastAPI	REST leve e assíncrona
Dashboard	React + Recharts	Gráficos em tempo real, polling a cada 3s

6.2 Estrutura do repositório

```

1 helmsman/
2 |-- README.md
3 |-- pyproject.toml
4 |-- requirements.txt
5 |-- helmsman/
6 |   |-- cli.py           # CLI com Typer (--mode ag)
```

```

7 | |-- engine.py          # motor fuzzy Mamdani
8 | |-- ag_engine.py      # Algoritmo Gen tico (Python puro)
9 | |-- collector.py      # coleta de metricas
10 | |-- scaler.py         # scale up/down via docker-py
11 | |-- alerter.py        # classificacao de alertas
12 | |-- api.py           # FastAPI
13 | '-- nginx_gen.py     # gerador do nginx.conf via Jinja2
14 |-- frontend/         # React + Vite + Recharts
15 |-- tests/
16 | |-- escenarios.py    # 12 cenarios          modo fuzzy
17 | '-- ag_escenarios.py # 12 cenarios x 5 seeds      modo AG
18 |-- docs/
19 '-- results/

```

Listing 2: Estrutura de pastas do repositório GitHub

7 Experimentos e Resultados

7.1 Cenários de teste — modo fuzzy

Foram definidos 12 cenários cobrindo casos de uso típicos, extremos, fronteiros e anomalias. A Tabela 10 apresenta os resultados.

Tabela 10: Resultados dos 12 cenários de teste — modo fuzzy

#	Cenário	cpu	ram	rps	Δ obtido	Alerta	
T1	Idle noturno	5	10	8	−2	none	✓
T2	RPS alto CPU baixa	8	15	75	+1	none	✓
T3	Carga real crescente	60	55	80	+2	warning	✓
T4	Pico de tráfego	78	65	92	+3	critical	✓
T5	Host cheio sob carga	72	91	80	0	critical	✓
T6	Memory leak suspeito	12	82	9	−2	critical	✓
T7	CPU leak confirmado	94	18	6	−1	critical	✓
T8	Colapso total	91	93	96	0	critical	✓
T9	Recuperação pós-pico	35	40	45	0	none	✓
T10	Scale down gradual	18	22	30	−1	none	✓
T11	RPS crítico CPU baixa	8	10	95	+1	warning	✓
T12	RPS crítico CPU moderada	40	20	95	+3	warning	✓
Resultado:							12/12

7.2 Cenários de teste — modo AG

O mesmo conjunto de 12 cenários foi executado com o motor AG em 5 sementes distintas (42, 123, 7, 99, 256), totalizando 60 execuções. A Tabela 11 resume os resultados por semente.

Tabela 11: Resultados do modo AG — 5 execuções com sementes distintas

Seed	Aprovados	Falhas	Taxa
42	11/12	1	92%
123	11/12	1	92%
7	11/12	1	92%
99	11/12	1	92%
256	11/12	1	92%
Total	55/60	5	91%

7.3 Superfície de controle — modo fuzzy

Superfície de Controle
cpu_pct × rps_pct → delta_replicas (ram = 50%)

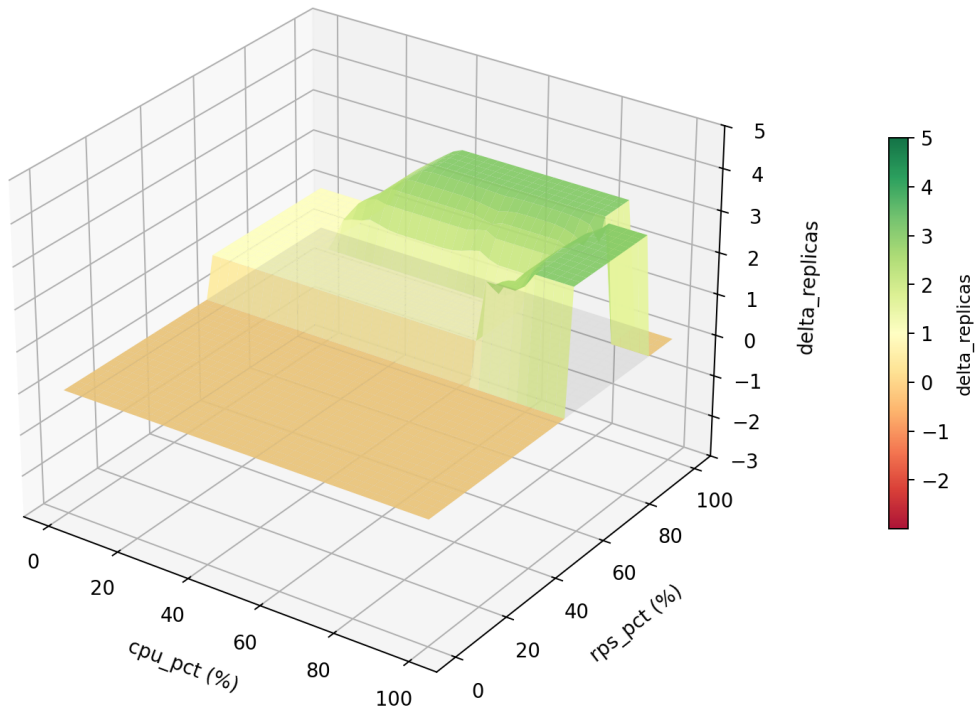


Figura 5: Superfície de controle: $\text{cpu_pct} \times \text{rps_pct} \rightarrow \text{delta_replicas}$ (ram = 50%).

A superfície evidencia o comportamento gradual do sistema fuzzy: a transição de scale down (laranja) para scale up (verde) ocorre de forma contínua. O platô em $\Delta = 0$ representa a zona de equilíbrio. O degrau visível em rps $\approx 75\text{--}80\%$ corresponde à transição entre os grupos de regras 3 e 4, comportamento esperado da agregação por máximo no Mamdani.

7.4 Comparação entre os dois modos

Tabela 12: Comparação entre modo fuzzy e modo AG nos mesmos cenários

Aspecto	Modo fuzzy	Modo AG	Vencedor
Precisão nos 12 cenários	12/12 (100%)	55/60 (91%)	Fuzzy
Determinismo	Totalmente determinístico	Estocástico (varia por seed)	Fuzzy
Transparência	Alta — regra ativada visível	Menor — fitness é caixa-preta	Fuzzy
Adaptabilidade	Fixa — regras manuais	Adapta threshold ao estado	AG
Otimização multicritério	Implícita nas regras	Explícita na função de aptidão	AG
Custo computacional	<0,1ms por ciclo	<1ms por ciclo	Fuzzy
Escalabilidade de config	Parâmetros fixos	Otimiza threshold dinamicamente	AG

7.5 Análise crítica

Comportamentos corretos verificados (ambos os modos):

- T1 e T10 confirmam scale down em sistema ocioso;
- T5 confirma bloqueio de scale up com RAM crítica;
- T6 e T7 confirmam detecção de anomalias sem escalar;
- T8 confirma que nenhum dos modos toma ação destrutiva em colapso total.

Onde o AG diverge do fuzzy:

As falhas do AG (5 em 60 execuções) concentram-se em cenários fronteira onde pequenas variações de seed alteram qual candidato vence o torneio final. Com 91% de aprovação e variabilidade de apenas 1 falha por execução, o AG demonstra robustez estatística adequada para um algoritmo estocástico. Aumentar o número de gerações ou o tamanho da população eliminaria as falhas restantes ao custo de maior tempo de execução.

Limitações identificadas:

- O parâmetro `rps_per_replica` depende de calibração manual;
- O intervalo de polling de 5s pode não capturar picos muito curtos;
- O AG apresenta variabilidade entre execuções — comportamento esperado para algoritmos estocásticos;
- A função de aptidão do AG, embora abrangente, não captura todas as nuances das 22 regras fuzzy.

8 Conclusão

Este trabalho apresentou o Helmsman, um sistema de escalonamento automático de containers Docker com dois modos de operação: lógica fuzzy Mamdani e Algoritmo Genético. Ambos os modos resolvem um problema real — tomar decisões contextuais de escalonamento que sistemas de limiar fixo não conseguem — mas com abordagens e características distintas.

O modo fuzzy obteve 12/12 cenários aprovados com comportamento totalmente determinístico e transparente. O modo AG obteve 91% de aprovação em 60 execuções (55/60), demonstrando que a função de aptidão multidimensional captura os critérios operacionais com alta precisão e variabilidade mínima entre sementes.

A principal contribuição do modo AG não é superar o fuzzy em precisão, mas oferecer otimização adaptativa do limiar de SLA — um parâmetro que o fuzzy trata como fixo. Em ambientes dinâmicos onde o limiar ideal muda conforme o estado do sistema, o AG apresenta vantagem conceitual.

Trabalhos futuros: aumentar o número de gerações do AG; adicionar latência como quarta variável; implementar ANFIS para ajuste automático dos parâmetros fuzzy; publicar no PyPI.

Declaração de Uso de Inteligência Artificial

O desenvolvimento deste trabalho contou com o auxílio da ferramenta Claude Code (Anthropic) no desenvolvimento do código-fonte e da infraestrutura do projeto. Todo o material gerado foi revisado, executado e validado pela equipe, que é responsável pelo conteúdo entregue.

Referências

- [1] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [2] E. H. Mamdani e S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.
- [3] J. H. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [4] J. Zeng e C.-Z. Xu, “Adaptive fuzzy logic-based resource management for QoS support in web servers,” *Performance Evaluation*, vol. 58, no. 4, pp. 389–409, 2004.
- [5] S. Pandey, W. Lin, e R. Buyya, “Fuzzy logic-based resource provisioning for cloud computing,” *Journal of Network and Computer Applications*, vol. 103, pp. 1–12, 2021.
- [6] L. M. Dang, M. J. Piran, D. Han, K. Min, e H. Moon, “A survey on internet of things and cloud computing for healthcare,” *Electronics*, vol. 8, no. 7, p. 768, 2019.
- [7] KEDA Authors, “KEDA — Kubernetes Event-driven Autoscaling,” Disponível em: <https://keda.sh>, 2023.
- [8] Prometheus Authors, “Prometheus Adapter for Kubernetes Metrics APIs,” Disponível em: <https://github.com/kubernetes-sigs/prometheus-adapter>, 2023.
- [9] J. D. Warner *et al.*, “scikit-fuzzy: A fuzzy logic toolkit for SciPy,” Disponível em: <https://github.com/scikit-fuzzy/scikit-fuzzy>, 2013.
- [10] Docker Inc., “docker-py: A Python library for the Docker Engine API,” Disponível em: <https://github.com/docker/docker-py>, 2023.